



UNIVERSITÀ DI PARMA

**Dipartimento di Scienze Matematiche, Fisiche Ed Informatiche
Corso Di Laurea In Scienze Informatiche**

Sentiment analysis su fake news Sentiment analysis on fake news

**Studente:
Andrea Agnetti**

Anno Accademico 2024/2025

Contenuto

1	Introduzione	1
2	Nozioni di base	1
2.1	LSTM	1
2.1.1	Architettura della LSTM	1
2.1.2	Aggiornamento dello stato della cella	2
2.1.3	Aggiornamento dello stato nascosto	3
2.2	Bidirectional LSTM (BLSTM)	3
3	Implementaizione	4
3.1	Preprocessing dei dati	4
3.2	Architettura del modello	5
3.3	Addestramento e ottimizzazione	6
4	Conclusioni	7
	Lista delle Figure	I
	Lista delle Tabelle	II
	Riferimenti	III

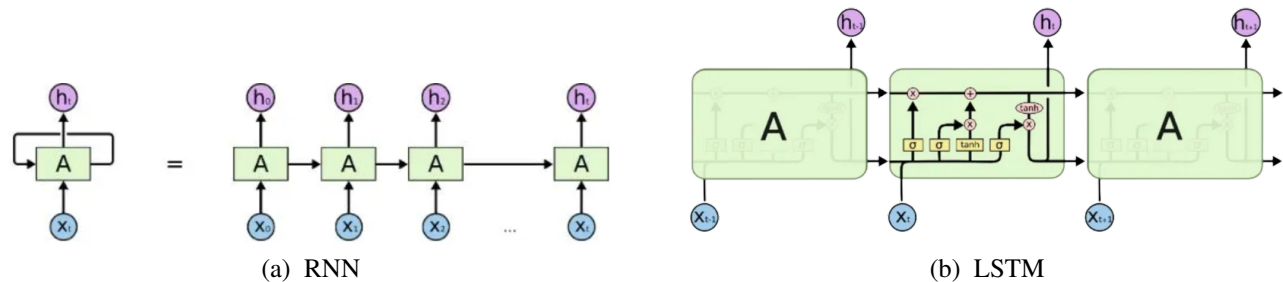


Figura 1: RNN e LSTM (Olah (2015))

1 Introduzione

La Sentiment Analysis, o Analisi del Sentimento, è un'area del Natural Language Processing (NLP) che si occupa di determinare l'atteggiamento di un autore verso un argomento o un prodotto, solitamente classificandolo come positivo, negativo o neutro. Nel mio caso, ho adattato questo approccio per un compito di classificazione binaria: identificare se un articolo di cronaca è "real" (reale) o "fake" (falso).

2 Nozioni di base

2.1 LSTM

Le Reti Neurali Ricorrenti (RNN) (Elman (1990)) soffrono del problema delle *long-term dependencies* (Bengio et al. (1994)), ovvero l'incapacità di catturare e utilizzare efficacemente le dipendenze tra elementi di una sequenza che sono separati da molti passi temporali.

Questo limite deriva principalmente dal *vanishing gradient problem* o dall'*exploding gradient problem*. Durante l'addestramento, infatti, i gradienti possono diventare estremamente piccoli (vanishing) o molto grandi (exploding) quando si propagano all'indietro nel tempo attraverso molti strati. Di conseguenza, le RNN tradizionali faticano a collegare informazioni presenti all'inizio della sequenza con eventi che si verificano molto più tardi.

2.1.1 Architettura della LSTM

Le Long Short-Term Memory (LSTM) (Hochreiter and Schmidhuber (1997)) sono usate per affrontare il problema del *vanishing gradient* nelle reti ricorrenti tradizionali (RNN). La loro peculiarità risiede nella capacità di memorizzare e gestire informazioni a lungo termine grazie a una struttura interna più sofisticata rispetto alle RNN semplici.

- **Cell State (stato della cella):** funge da "nastro trasportatore" che scorre lungo l'intera catena,

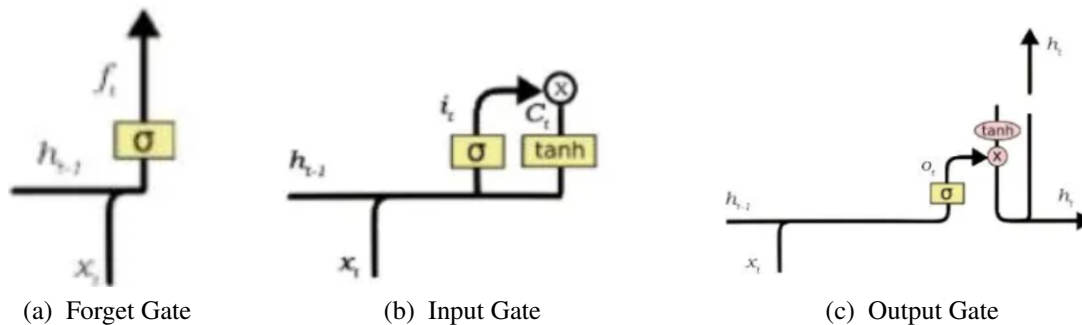


Figura 2: Gates LSTM

trasportando informazioni rilevanti. Può mantenere dati per lunghi periodi di tempo senza alterazioni, a meno che non vengano modificati esplicitamente da un gate. Rappresenta la memoria a lungo termine della rete.

- **Gates**: meccanismi che regolano il flusso delle informazioni. Ogni gate è costituito da un livello neurale con funzione di attivazione sigmoide (output tra 0 e 1) che decide quante informazioni devono passare. In particolare:
 - **Forget Gate (porta dell'oblio)**: decide quali informazioni dello stato precedente devono essere scartate.
 - **Input Gate (porta di input)**: regola quali nuove informazioni devono essere memorizzate nello stato della cella.
 - **Output Gate (porta di uscita)**: determina, sulla base dello stato della cella e dell'input corrente, quali informazioni devono essere prodotte come output.

Grazie a questa architettura, le LSTM sono in grado di memorizzare, dimenticare e aggiornare selettivamente le informazioni, mantenendo il contesto a lungo termine e riducendo l'impatto del vanishing gradient. Per questo motivo risultano particolarmente efficaci in compiti come l'analisi del sentimento, dove il significato di una parola può dipendere da un'altra apparsa molto tempo prima nella sequenza.

2.1.2 Aggiornamento dello stato della cella

Il processo di aggiornamento dello stato della cella avviene attraverso un'interazione bilanciata tra conservazione e integrazione di nuove informazioni. Il **forget gate** (2a), ricevendo in input lo stato nascosto precedente e l'input corrente, produce un vettore di valori tra 0 e 1 che agiscono come maschere: valori vicini a 0 annullano informazioni ormai irrilevanti, mentre valori vicini a 1 le preservano.

Parallelamente, l'**input gate** (2b) regola l'ingresso di nuove informazioni. Per farlo, utilizza due funzioni di attivazione: una sigmoide, che decide quali posizioni della memoria aggiornare, e una tangente iperbolica, che genera i nuovi valori da memorizzare (in un intervallo compreso tra -1 e 1).

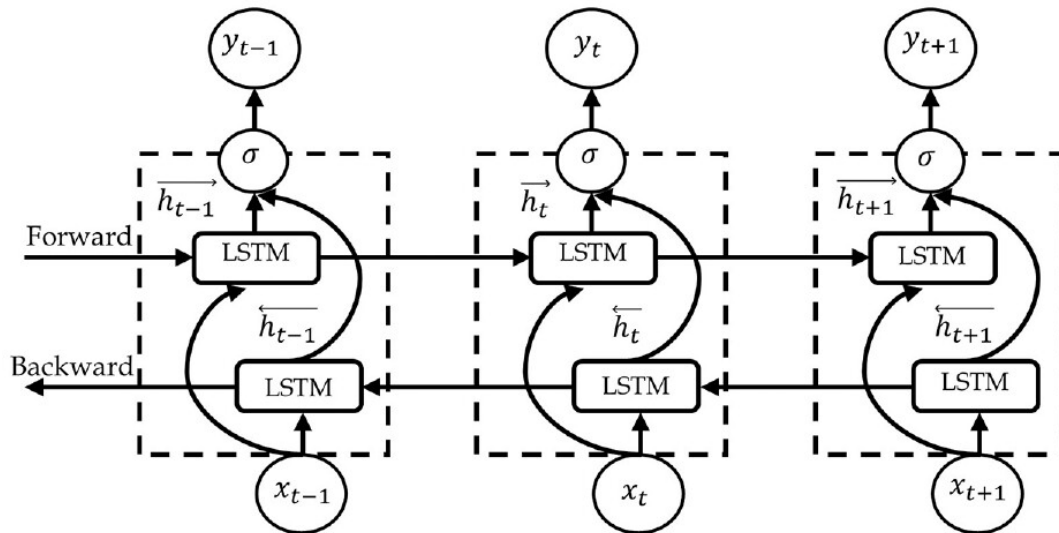


Figura 3: LSTM bidirezionale (immagine da Degachi and Ouni (2025))

La combinazione di questi due risultati fornisce il contributo che verrà aggiunto allo stato della cella.

Infine, il nuovo stato della cella si ottiene combinando il contributo del forget gate (che preserva parte del vecchio contenuto) con quello dell'input gate (che inserisce le nuove informazioni). Questo aggiornamento incrementale consente alla LSTM di bilanciare memoria e adattamento al contesto.

2.1.3 Aggiornamento dello stato nascosto

Lo stato nascosto, che rappresenta la memoria a breve termine e viene propagato tra i vari passi temporali, è calcolato a partire dall'**output gate** (2c). Quest'ultimo utilizza una funzione sigmoide applicata all'input corrente e allo stato nascosto precedente per selezionare quali informazioni trasmettere. Il risultato viene poi modulato dal nuovo stato della cella, trasformato tramite una tangente iperbolica. La moltiplicazione elemento per elemento tra questi due vettori produce lo stato nascosto aggiornato, che verrà utilizzato come input per il passo successivo della rete.

Questa separazione tra memoria a lungo termine (stato della cella) e memoria a breve termine (stato nascosto) costituisce uno degli aspetti più innovativi delle LSTM, e consente loro di mantenere informazioni anche su intervalli temporali molto ampi senza perdere la capacità di adattarsi dinamicamente al contesto immediato.

2.2 Bidirectional LSTM (BLSTM)

Le LSTM bidirezionali (*Bidirectional LSTM*, BLSTM) (Graves et al. (2005)) rappresentano un'estensione naturale delle LSTM standard, pensata per arricchire ulteriormente il contesto a disposizione del modello.

In una LSTM tradizionale, la sequenza viene elaborata solo in una direzione (tipicamente dall'inizio alla fine), e il modello dispone quindi solo del contesto passato. Le BLSTM, invece, utilizzano due strati paralleli:

- **Forward Pass:** un primo strato LSTM elabora la sequenza da sinistra a destra, catturando il contesto passato.
- **Backward Pass:** un secondo strato LSTM elabora la stessa sequenza da destra a sinistra, catturando il contesto futuro.

Gli stati nascosti dei due strati vengono successivamente combinati, fornendo al modello una rappresentazione più ricca e completa. Questo approccio risulta particolarmente vantaggioso in compiti di elaborazione del linguaggio, dove il significato di una parola dipende spesso sia da ciò che la precede sia da ciò che la segue, come accade nella traduzione automatica o nel riconoscimento del parlato.

3 Implementazione

L'obiettivo del progetto è la costruzione di un modello di deep learning capace di distinguere tra notizie vere e notizie false (*fake news detection*). Per questo scopo è stato utilizzato il dataset **WELFake**, che contiene un totale di 72.134 articoli (35.028 reali e 37.106 falsi), ottenuti combinando quattro dataset popolari (*Kaggle*, *McIntire*, *Reuters*, *BuzzFeed Political*) al fine di aumentare la varietà e ridurre il rischio di *overfitting*.

Il dataset è composto da quattro colonne:

- **Serial number:** indice numerico progressivo;
- **Title:** titolo della notizia;
- **Text:** contenuto testuale della notizia;
- **Label:** etichetta binaria (0 = notizia falsa, 1 = notizia vera).

Nel file CSV originale sono presenti 78.098 record, ma solo 72.134 sono stati utilizzati a seguito di pulizia e rimozione di valori mancanti.

3.1 Preprocessing dei dati

Le fasi di preprocessing hanno incluso:

1. **Pulizia del testo:** trasformazione in minuscolo, rimozione di link, punteggiatura, caratteri numerici e markup HTML.
2. **Unione dei campi:** concatenazione del titolo e del contenuto in un'unica colonna testuale.
3. **Tokenizzazione e vettorizzazione:** utilizzo di un tokenizer Keras limitato alle 10.000 parole più frequenti del dataset.

4. **Padding:** uniformazione delle sequenze a lunghezza 500 token per gestire articoli di diversa dimensione.
5. **Suddivisione del dataset:** 70% training, 15% validation, 15% test.

3.2 Architettura del modello

Il modello è stato implementato utilizzando Keras e si basa su una rete neurale ricorrente bidirezionale con strati LSTM. L'idea alla base di questa architettura è combinare la capacità delle LSTM di catturare dipendenze a lungo termine all'arricchimento contestuale fornito dall'elaborazione bidirezionale. In questo modo, il modello può considerare sia le informazioni precedenti che quelle successive all'interno di una sequenza di parole, caratteristica particolarmente utile nell'elaborazione del linguaggio naturale, dove il significato di un termine può dipendere dal contesto globale della frase o addirittura del paragrafo.

La Tabella 1 riassume i livelli costitutivi della rete.

Layer (type)	Output Shape	Param #
Embedding (128-dim)	(None, 500, 128)	1,280,000
Bidirectional LSTM	(None, 500, 128)	98,816
Bidirectional LSTM	(None, 128)	98,816
Dropout (0.5)	(None, 128)	0
Dense (sigmoid)	(None, 1)	129

Tabella 1: Architettura della rete neurale per la classificazione delle notizie.

Embedding Layer. Il primo livello è un **Embedding** che proietta le sequenze di parole in uno spazio vettoriale denso di 128 dimensioni. In questo spazio, parole semanticamente simili vengono collocate in posizioni vicine, favorendo l'apprendimento delle relazioni contestuali. L'uso dell'embedding riduce inoltre la dimensionalità rispetto a una rappresentazione *one-hot*, permettendo di catturare somiglianze lessicali e semantiche.

Primo strato Bidirectional LSTM. Il primo livello **Bidirectional LSTM**, composto da 64 unità in ciascuna direzione (forward e backward), restituisce sequenze complete come output. Ciò significa che per ogni parola della frase viene generata una rappresentazione contestuale che tiene conto sia delle parole precedenti che di quelle successive. Questo strato è utile per modellare le relazioni locali e preservare informazioni dettagliate lungo l'intera sequenza (500 token). Ad esempio, nel contesto delle fake news, questo livello può riconoscere pattern ricorrenti nel linguaggio o espressioni tipiche che si ripetono in più punti del testo.

Secondo strato Bidirectional LSTM. Il secondo livello **Bidirectional LSTM**, anch'esso con 64 unità, ha il compito di comprimere le informazioni ottenute dallo strato precedente in un'unica rappresentazione finale. A differenza del primo, qui non vengono restituite tutte le sequenze intermedie, ma solo lo stato nascosto finale (dimensione 128). Questo passaggio consente di sintetizzare l'intero articolo in un vettore denso, che funge da rappresentazione globale del testo. In altre parole, se il primo strato serve a catturare le relazioni dettagliate parola per parola, il secondo ha un ruolo più riassuntivo e fornisce una visione complessiva della notizia.

Dropout Layer. Per ridurre il rischio di *overfitting*, viene inserito un livello di **Dropout** con tasso 0.5. Durante l'addestramento, questo livello disattiva casualmente la metà dei neuroni del livello precedente a ogni iterazione, costringendo la rete a non dipendere da specifiche combinazioni di neuroni e a imparare rappresentazioni più robuste. Questa tecnica si rivela particolarmente efficace nei modelli di NLP, dove il rischio di memorizzare pattern non generalizzabili è elevato.

Dense Layer. Infine, lo strato **Dense** con un singolo neurone e funzione di attivazione *sigmoide* produce un output scalare compreso tra 0 e 1, interpretabile come probabilità che la notizia sia reale. Se il valore è vicino a 0, la notizia viene classificata come *fake*, mentre se è vicino a 1 viene classificata come *real*.

Nel complesso, l'architettura è progettata per bilanciare dettaglio e sintesi: il primo strato LSTM estrae le relazioni locali, il secondo le riassume in una rappresentazione compatta, il dropout ne garantisce la generalizzazione e il livello denso finale consente la classificazione binaria.

3.3 Addestramento e ottimizzazione

Il modello è stato addestrato utilizzando l'ottimizzatore **Adam**, una scelta molto diffusa nelle reti neurali per via della sua capacità di combinare i vantaggi di *AdaGrad* e *RMSProp*. Adam adatta dinamicamente il tasso di apprendimento per ciascun parametro della rete, accelerando la convergenza e garantendo maggiore stabilità durante l'addestramento.

Come funzione di costo è stata impiegata la **binary cross-entropy** (1), particolarmente adatta ai problemi di classificazione binaria. Essa misura la distanza tra la distribuzione prevista dal modello e quella reale, penalizzando fortemente le classificazioni errate con alta confidenza. In questo modo la rete viene spinta a produrre probabilità ben calibrate, requisito fondamentale per distinguere notizie *fake* da notizie *real*.

$$L = - \sum_{i=1}^m [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)] \quad (1)$$

La metrica principale di valutazione è stata l'**accuratezza**, che fornisce una misura intuitiva della proporzione di esempi correttamente classificati. Sebbene non tenga conto degli eventuali squilibri di classe, in questo caso si è rivelata adeguata grazie alla distribuzione bilanciata del dataset.

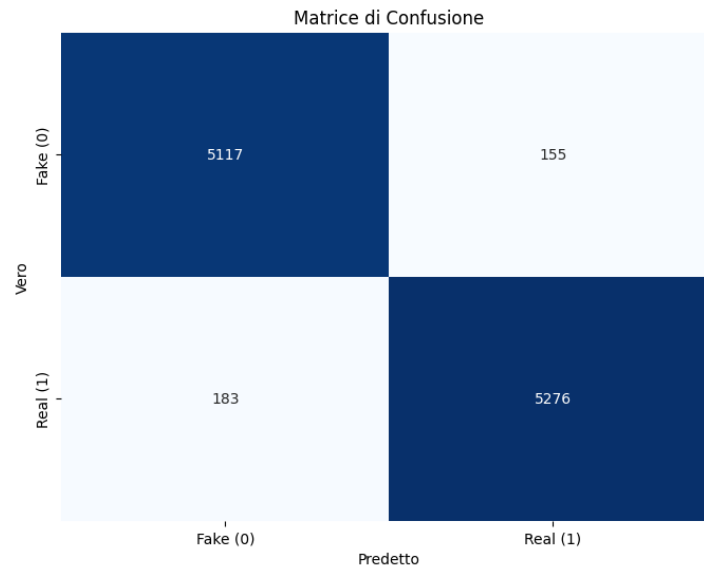


Figura 4: Esempio di matrice di confusione

Per migliorare la capacità di generalizzazione del modello e ridurre il rischio di overfitting sono stati introdotti due meccanismi di regolarizzazione:

- **Early Stopping:** l'addestramento viene interrotto automaticamente se la *validation loss* non mostra miglioramenti per 3 epoche consecutive. Questo approccio evita che la rete continui a imparare pattern specifici del set di training, prevenendo la perdita di generalizzazione.
- **Model Checkpoint:** a ogni epoca viene salvata una copia del modello corrispondente alle migliori prestazioni sulla validazione. In questo modo si conserva sempre la versione più efficace della rete, indipendentemente da eventuali fluttuazioni negative nelle epoche successive.

Nel complesso, la combinazione di ottimizzazione adattiva, funzione di costo appropriata e strategie di regolarizzazione ha garantito un addestramento stabile ed efficiente, conducendo a un modello robusto e in grado di generalizzare correttamente anche su dati mai visti.

4 Conclusioni

Il modello finale è stato valutato sul test set tramite le principali metriche di classificazione binaria: accuratezza, precisione, recall, F1-score e matrice di confusione (4).

In particolare, la **matrice di confusione** ha permesso di evidenziare la capacità del modello di distinguere correttamente le notizie reali da quelle false, mentre l'analisi delle curve di *loss* e *accuracy* (training vs validation) ha mostrato un buon livello di generalizzazione, senza fenomeni marcati di overfitting.

Lista delle Figure

1	RNN e LSTM (Olah (2015))	1
2	Gates LSTM	2
3	LSTM bidirezionale (immagine da Degachi and Ouni (2025))	3
4	Esempio di matrice di confusione	7

Lista delle Tabelle

1 Architettura della rete neurale per la classificazione delle notizie. 5

Riferimenti

- Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE transactions on neural networks*, 5(2), 157–166.
- Degachi, O. & Ouni, K. (2025). An attention-augmented bidirectional lstm-based encoder–decoder architecture for electrocardiogram heartbeat classification. *Transactions of the Institute of Measurement and Control*, 47(3), 506–515.
- Elman, J. L. (1990). Finding structure in time. *Cognitive science*, 14(2), 179–211.
- Graves, A., Fernández, S., & Schmidhuber, J. (2005). Bidirectional lstm networks for improved phoneme classification and recognition. In *International conference on artificial neural networks*, pages 799–804. Springer.
- Hochreiter, S. & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8), 1735–1780.
- Olah, C. (2015). Understanding LSTM networks. <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>. Accessed: [September 16, 2025].